

Node超时属性设计方案

Node超时属性设计方案

版本	修订人	日期	内容
v1.0	王亚雷	2025年04月25日	初版
v1.1	王亚雷	2025年04月25日	根据反馈调整设计

1. 前期分析

1.1 背景简介

GBF框架是一个业务流程编排框架，其中NodeConfigBuilder用于构建节点配置。目前框架支持异步执行节点，但缺少针对单个节点的超时控制能力，需要增强框架的超时管理能力。

1.2 目标收益

1. 提升框架的可控性，支持针对单个异步节点的超时控制
2. 优化资源利用，避免长时间等待异步任务导致资源浪费
3. 增强框架的错误处理能力，提供统一的超时处理机制

1.3 需求分析

1.3.1 功能需求

1. 在NodeConfigBuilder中增加timeout属性，用于设置异步节点的超时时间
2. 提供async(timeout)方法，支持同时设置异步开关和超时时间
3. 确保timeout参数能够传递到执行环节并生效
4. 在异步Node结果中根据timeout判断是否需要等待future结果
5. 超时时间从执行开始计算，全局一致，超时后直接返回超时失败

1.3.2 技术需求

1. 保持与现有异步执行机制的兼容性
2. 超时触发后的错误处理需统一并明确

3. 所有调用方法对外接口保持稳定，不能破坏现有调用方式

1.4 实现分析

1.4.1 前期调研

1.4.1.1 产品调研

框架目前已有全局的异步节点超时配置（`Properties.getProcessAsyncNodeTimeout()`），但缺少针对单个节点的超时控制能力。

1.4.1.2 技术调研

当前框架中的异步执行模式：

1. `NodeConfig`中有`asyncEnable`属性控制异步执行
2. `ProcessNodeInvoker`在执行时根据`asyncEnable`决定同步或异步执行
3. 异步执行通过`CompletableFuture`实现，结果封装在`FutureResult`中
4. `FutureResult.getResult()`方法从`CompletableFuture`获取结果，有内置的超时处理

1.4.2 差异分析

1.4.2.1 产品差异

1. 现有实现只支持全局统一的异步节点超时配置
2. 新方案将支持针对单个节点的超时时间配置，增强精细化控制能力

1.4.2.2 技术差异

1. 现有`NodeConfig`类中不包含`timeout`属性
2. 现有`FutureResult.getResult()`方法使用全局配置的超时时间
3. 新方案将在`NodeConfig`中增加`timeout`属性，并在`FutureResult`中基于开始执行时间计算过期时间点

1.4.3 实现结论

基于现有框架的架构，我们可以通过在`NodeConfig`中增加`timeout`属性，并在异步执行路径上传递和使用该属性来实现需求。`FutureResult`将在构造时计算过期时间点，确保全局一致的超时判断。

2. 方案设计

2.1 整体方案

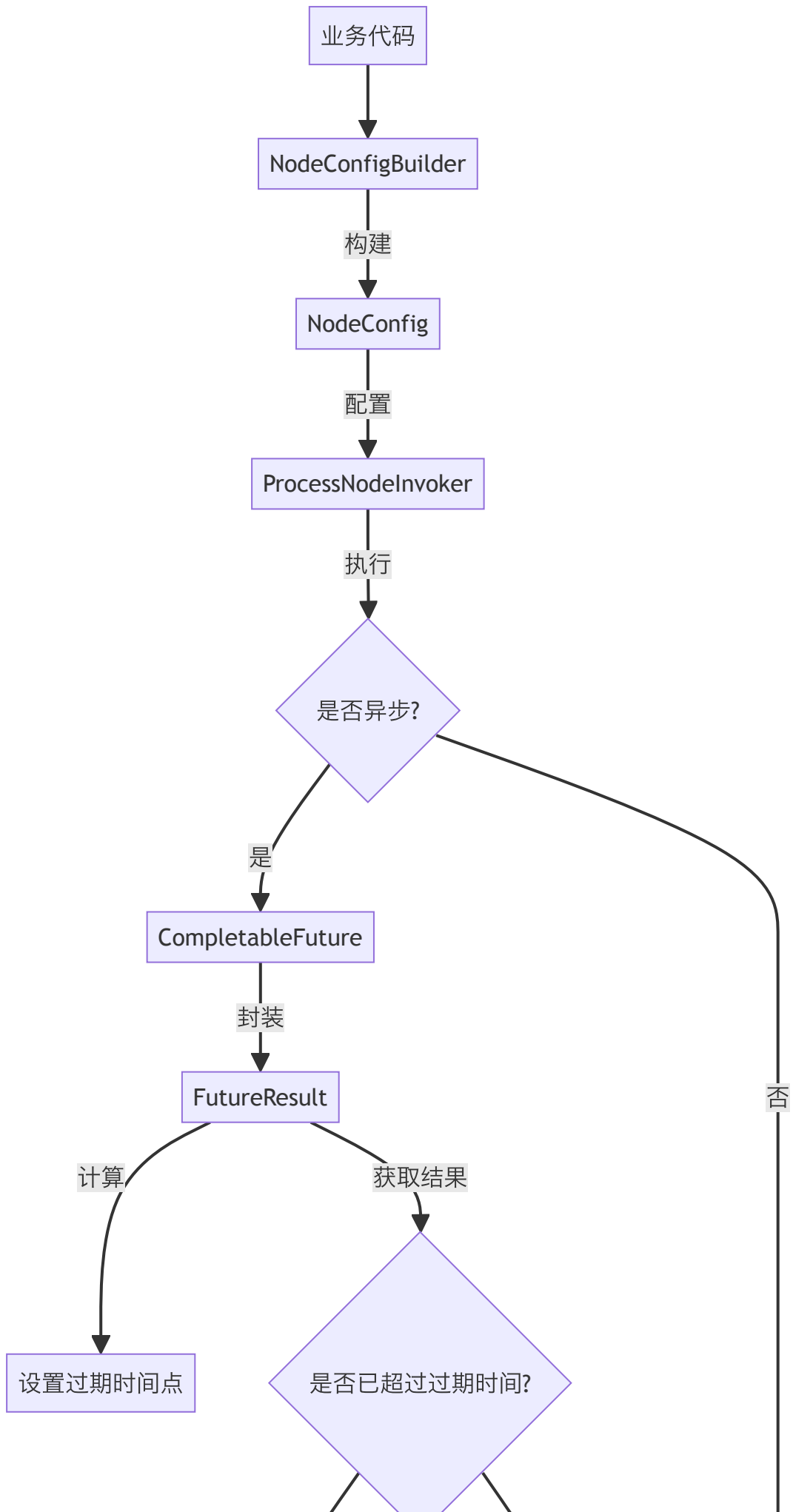
2.1.1 业务架构

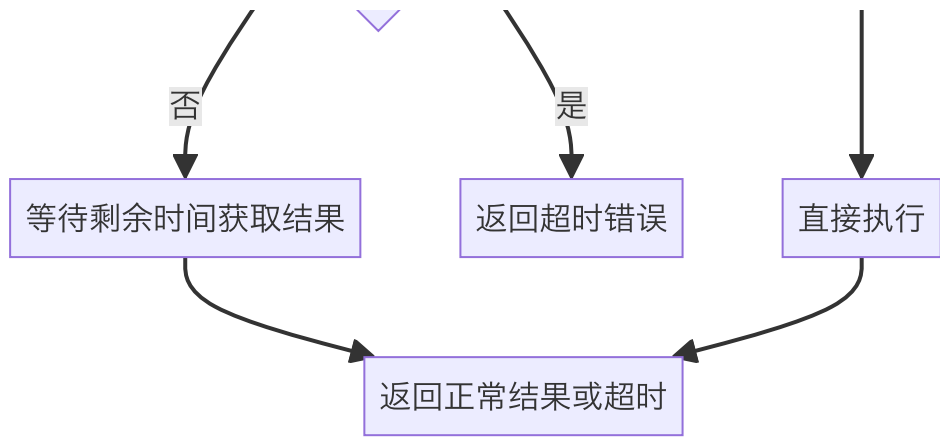
在现有GBF框架的Node节点配置和执行流程中增加超时控制，主要涉及以下业务流程：

1. 节点配置构建阶段：支持设置超时时间
2. 节点执行阶段：传递超时时间参数
3. 结果获取阶段：基于过期时间点进行判断

2.1.2 系统架构

系统架构保持不变，在现有组件基础上增加超时属性和相关处理逻辑：





2.2 关键方案

2.2.1 数据模型

需要修改的核心数据模型：

1. BaseNodeConfigBuilder类新增属性：



复制代码

```
// 超时时间（毫秒）  
protected Long timeout;
```

2. NodeConfig类新增属性：



复制代码

```
// 超时时间（毫秒）  
private Long timeout;
```

3. FutureResult类改进：

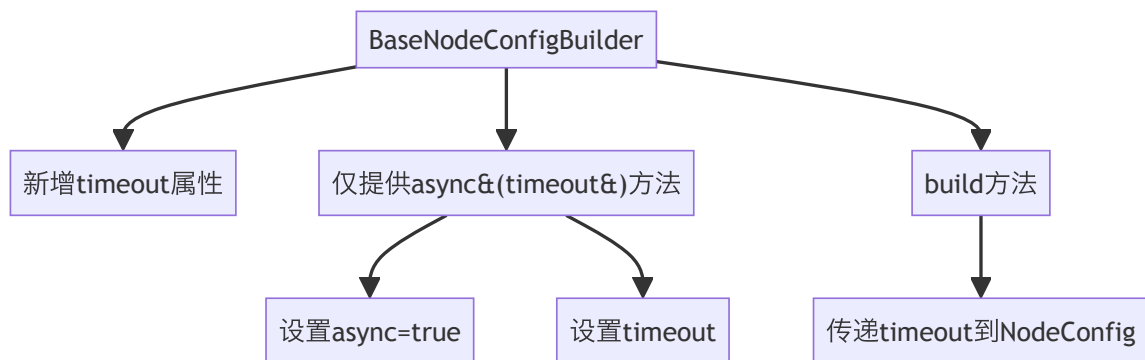


复制代码

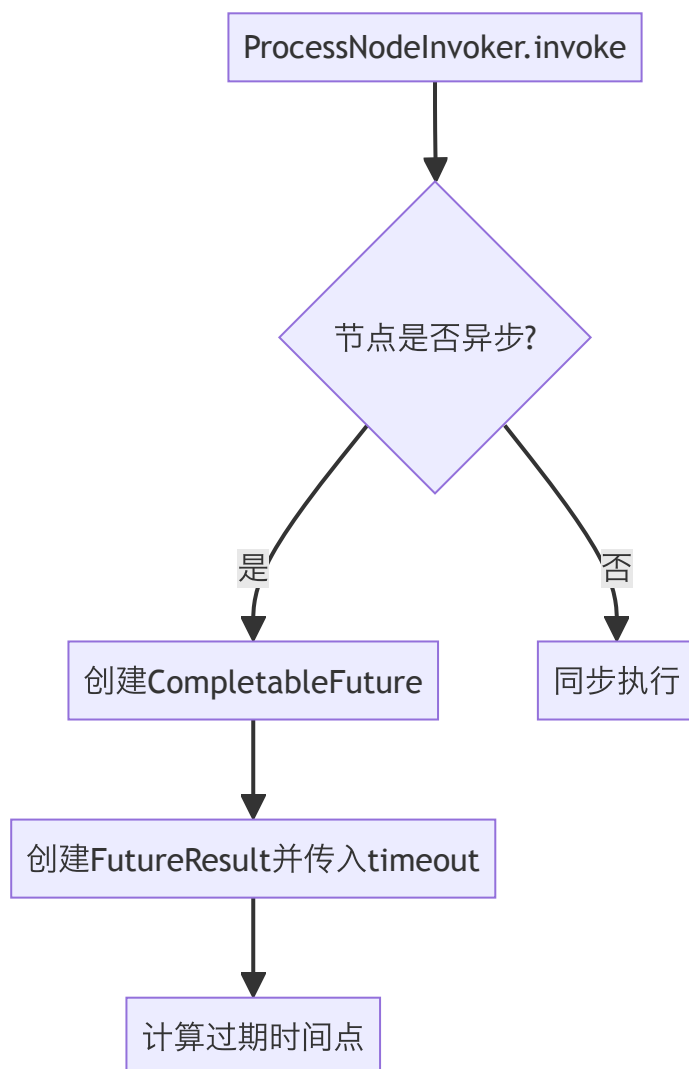
```
// 过期时间点（毫秒时间戳）  
private Long deadlineTimestamp;
```

2.2.2 关键逻辑

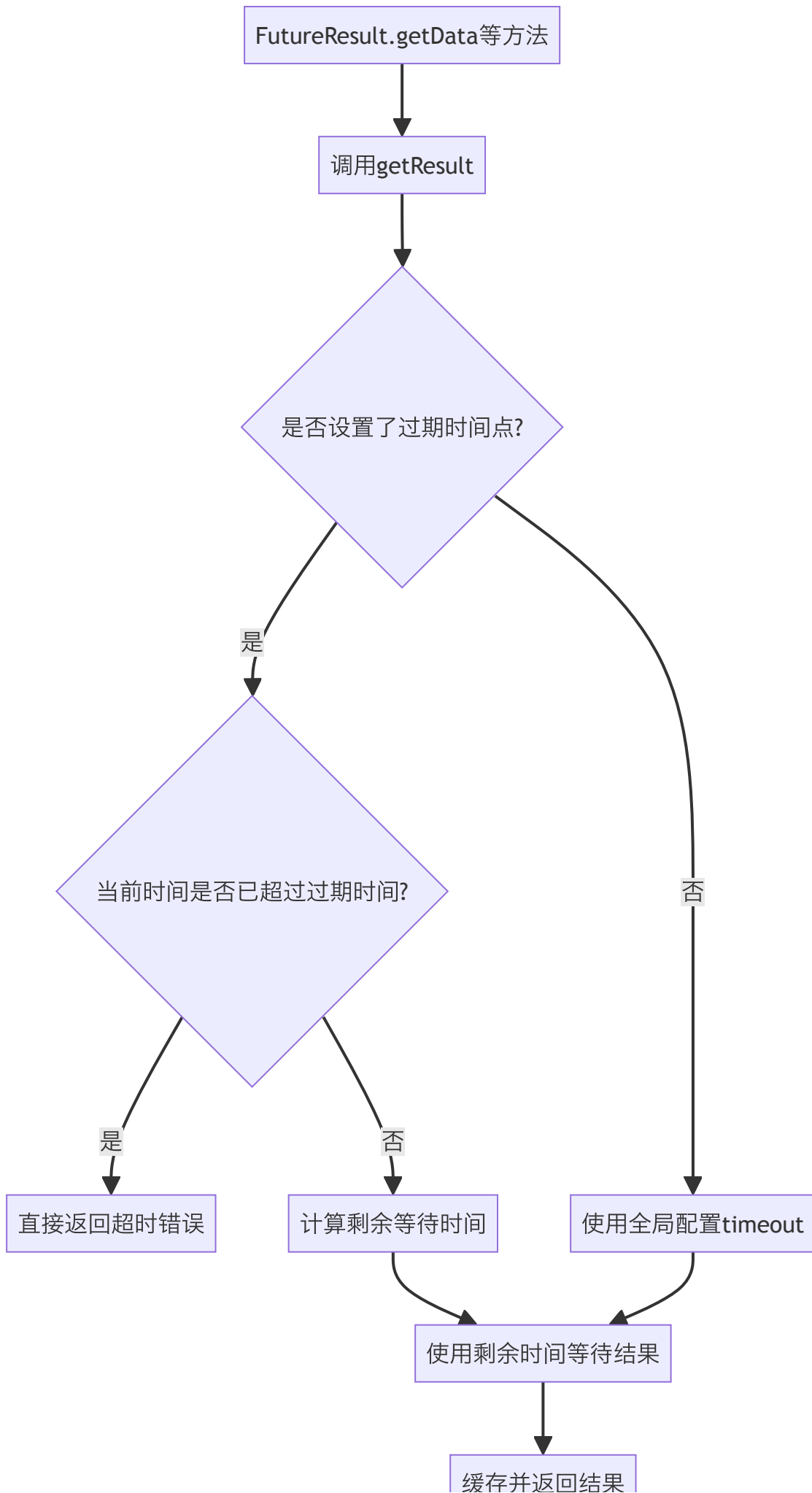
1. BaseNodeConfigBuilder中新增async(timeout)方法：



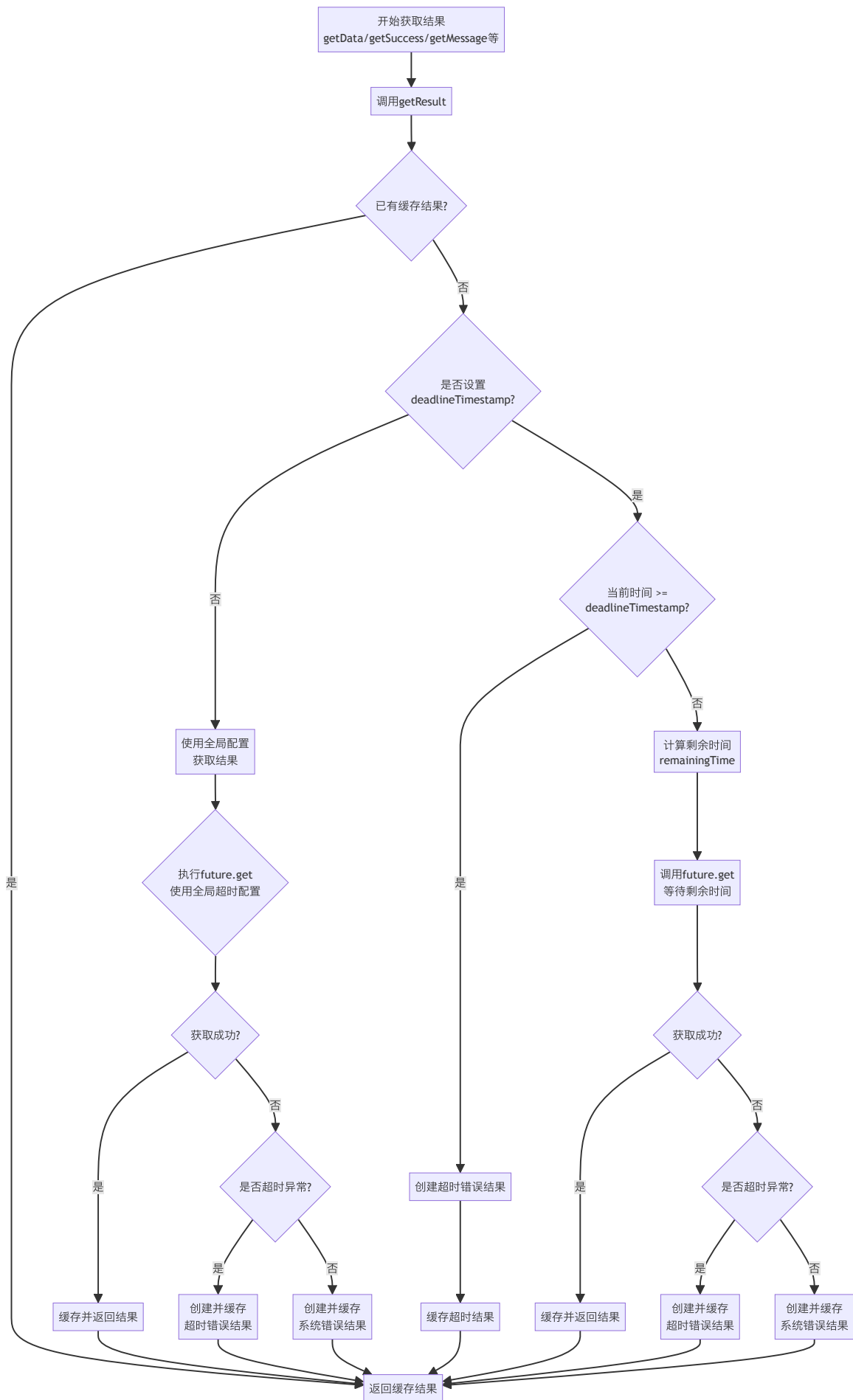
2. ProcessNodeInvoker中异步执行时传递timeout:



3. FutureResult中基于过期时间点判断:



4. FutureResult结果获取详细流程：



2.2.3 稳定性

1. 兼容性保障：

- 现有不设置timeout的调用方式保持不变，使用全局配置的超时时间
- 新增的timeout属性为可选属性，不会影响现有代码

2. 错误处理：

- 超时情况统一使用现有的ErrorCode.TIMEOUT错误码
- 一旦超时，后续所有结果获取操作都直接返回超时失败，不再重新等待

3. 详细设计

3.1 接口协议

本次修改主要是内部API的调整，不涉及对外接口变更。新增的方法如下：

1. BaseNodeConfigBuilder新增方法：

▼ 复制代码

```
public BaseNodeConfigBuilder async(Long timeout) {
    this.async = true;
    this.timeout = timeout;
    return this;
}
```

3.2 代码实现

具体需要修改的代码如下：

1. BaseNodeConfigBuilder.java

▼ 复制代码

```
// 增加timeout属性
protected Long timeout;

// 仅提供async(timeout)方法，移除单独的timeout方法
public BaseNodeConfigBuilder async(Long timeout) {
    this.async = true;
    this.timeout = timeout;
    return this;
}

// 修改build方法，传递timeout
```

```

public NodeConfig build() {
    // 原有验证逻辑保持不变
    if (this.nodeBeanName == null || "".equals(this.nodeBeanName)) {
        throw new ProcessException("NodeConfigBuilder.nodeBeanName
can't empty ");
    }
    String nodeBeanName = this.nodeBeanName.trim();

    // 传递timeout参数
    NodeConfig nodeConfig = new NodeConfig(nodeBeanName, alias,
predicatePoint, preJoinPoint, postJoinPoint, async, notWait, timeout);
    if (failBreak) {
        List<EndPoint> postJoinPoint = nodeConfig.getPostJoinPoint();
        List<EndPoint> endPointList = new ArrayList<>();
        endPointList.add(FailBreakEndPoint.INSTANCE);
        endPointList.addAll(postJoinPoint);
        nodeConfig.setPostJoinPoint(endPointList);
    }
    return nodeConfig;
}

```

2. NodeConfig.java

复制代码

```

// 增加timeout属性
private Long timeout;

// 修改现有构造函数，增加timeout参数
public NodeConfig(String nodeBeanName, String alias, PredicatePoint
predicatePoint,
                    List<JoinPoint> preJoinPoint, List<EndPoint>
postJoinPoint,
                    Boolean asyncEnable, Boolean notWait, Long timeout) {
    this.nodeBeanName = nodeBeanName;
    this.alias = alias;
    this.predicatePoint = predicatePoint;
    this.preJoinPoint = preJoinPoint;
    this.postJoinPoint = postJoinPoint;
    this.asyncEnable = asyncEnable;
    this.notWait = notWait;
    this.timeout = timeout;
}

```

3. ProcessNodeInvoker.java

复制代码

```
// 修改异步执行部分，传递timeout
if (Boolean.TRUE.equals(node.getAsyncEnable())) {
    Map<String, Context> contextMap = new HashMap<>
(ProcessPipeline.get());
    final Request finalRequest = request;
    Thread currentThread = Thread.currentThread();
    CompletableFuture<Result<Response>> resultCompletableFuture =
CompletableFuture.supplyAsync(() -> {
        Result<Response> res;
        try {
            /**
             * executorServiceSupplier 处理 RpcContext_inner、
DefaultSession、MDC的传递
            */
            ProcessPipeline.set(contextMap);
            res = post(node.getPostJoinPoint(), doInvoke(finalRequest,
node));
        } finally {
            // ThreadPoolExecutor.CallerRunsPolicy 拒绝策略兼容
            if (Thread.currentThread().getId() !=
currentThread.getId()) {
                ProcessPipeline.clear();
            }
        }
        return res;
    }, executorServiceSupplier.get());

    // 创建FutureResult时直接传入超时时间，在构造中计算过期时间点
    result = new FutureResult<>(resultCompletableFuture,
node.getTimeout());
} else {
    result = post(node.getPostJoinPoint(), doInvoke(request, node));
}
```

4. FutureResult.java

复制代码

```
// 使用过期时间点而非超时时长
private Long deadlineTimestamp;

// 修改构造函数，计算过期时间点
public FutureResult(CompletableFuture<Result<T>>
resultCompletableFuture, Long timeout) {
    this.resultCompletableFuture = resultCompletableFuture;
    if (timeout != null && timeout > 0) {
```

```

        this.deadlineTimestamp = System.currentTimeMillis() + timeout;
    }
}

// 兼容现有调用
public FutureResult(CompletableFuture<Result<T>>
resultCompletableFuture) {
    this(resultCompletableFuture, null);
}

// 缓存结果，避免重复获取
private volatile Result<T> cachedResult;

// 超时结果生成方法
private Result<T> getTimeoutResult() {
    if (cachedResult == null) {
        synchronized (this) {
            if (cachedResult == null) {
                cachedResult =
NodeResult.fail(ErrorCode.TIMEOUT.getValue(), "Node execution timeout",
new TimeoutException("Node execution timeout"));
            }
        }
    }
    return cachedResult;
}

// 修改getResult()方法，根据过期时间点计算剩余超时时间
protected Result<T> getResult() {
    if (deadlineTimestamp != null) {
        long currentTime = System.currentTimeMillis();
        if (currentTime >= deadlineTimestamp) {
            // 已超时，直接返回超时错误
            return getTimeoutResult();
        }
        // 计算剩余时间
        long remainingTime = deadlineTimestamp - currentTime;
        return getResult(remainingTime, TimeUnit.MILLISECONDS);
    }
    // 无过期时间点，使用全局配置
    return getResult(Properties.getProcessAsyncNodeTimeout(),
TimeUnit.MILLISECONDS);
}

// 修改getResult(long, TimeUnit)方法，支持结果缓存

```

```

public Result<T> getResult(long timeout, @NotNull TimeUnit unit) {
    // 如果已有缓存结果，直接返回
    if (cachedResult != null) {
        return cachedResult;
    }

    synchronized (this) {
        if (cachedResult != null) {
            return cachedResult;
        }

        try {
            cachedResult = resultCompletableFuture.get(timeout, unit);
            return cachedResult;
        } catch (InterruptedException | ExecutionException |
TimeoutException e) {
            if (e instanceof TimeoutException) {
                cachedResult =
NodeResult.fail(ErrorCode.TIMEOUT.getValue(), e.getMessage(), e);
            } else {
                cachedResult =
NodeResult.fail(ErrorCode.SYSTEM_ERROR.getValue(), e.getMessage(), e);
            }
            return cachedResult;
        }
    }
}

```

4. 其他

4.1 测试范围

测试需要覆盖以下场景：

1. 异步节点不设置timeout，使用全局配置超时时间
2. 异步节点设置timeout，使用自定义超时时间
3. 异步节点超时后，再次获取结果直接返回超时错误
4. 异步节点正常完成，获取结果正常返回
5. 同步节点设置timeout（应该无效）

4.2 成本评估

1. 开发工作量：2人天
2. 测试工作量：1人天

3. 上线风险：低（无对外接口变更，仅内部逻辑调整）

4.3 沟通过程

1. 与框架开发团队确认设计方案
2. 与使用方沟通新功能使用方式

4.4 实施计划

1. 开发阶段：2天
2. 测试阶段：1天
3. 发布阶段：1天

4.5 遗留内容

暂无

4.6 后续规划

1. 考虑对更多异步执行场景增加超时控制
2. 优化超时监控和告警机制

5. 附录

5.1 名词解释

- NodeConfig：节点配置，描述节点的执行方式和参数
- FutureResult：异步执行结果，封装CompletableFuture
- timeout：超时时间，单位为毫秒
- deadlineTimestamp：过期时间点，超时判断的基准时间戳

5.2 参考文档

1. GBF框架开发文档
2. Java CompletableFuture API文档